

## Modifiche dell'architettura

### 1. Database MySQL

- Modificata la tabella user con l'aggiunta di due nuove colonne "high\_value" e "low\_value"

### 2. Server gRPC

- Aggiunto il supporto per l'inserimento dei parametri "low\_value" e "high\_value". In particolare, è stata aggiunta un'ulteriore rpc al servizio "ManageUser"
- Implementato il pattern "Command Query Responsibility Segregation"

### 3. Client gRPC

- Aggiunto il supporto per l'inserimento dei parametri "low\_value" e "high\_value"

### 4. Data Collector

- Questo componente è il producer dei messaggi relativi al topic "to-alert-system". In particolare, verrà prodotto un messaggio di notifica ogni qual volta i dati nel Database vengono aggiornati. Tale messaggio viene inviato ai broker Kafka in maniera sincrona, bloccando l'esecuzione in attesa che il messaggio venga inviato.
- Implementato il pattern "Command Query Responsibility Segregation"

## Aggiunte all'architettura

### 1. Alert System

- Questo componente è consumer del topic "to-alert-system" e producer sul topic "to-notifier". Al ricevimento di un messaggio, analizza il database e per ciascun profilo utente in cui il valore del ticker risulta superiore a "high-value" o inferiore a "low-value" (se definiti), genera un messaggio come producer e lo pubblica sul topic "to-notifier". Il messaggio è composto dai seguenti parametri: <email, ticker, condizione di superamento soglia> ed è inviato in maniera sincrona. Successivamente all'invio di tale messaggio, verrà effettuato un commit manuale sui messaggi da esso ricevuti (come consumer). In questo modo siamo certi che nessun messaggio verrà mai perso nel caso in cui si verifichi un fault. Nel caso peggiore, ossia quello in cui il fault

si verifichi poco prima dell'invio del commit, verranno inviati due messaggi uguali. In questo modo siamo certi che l'utente non perda alcun aggiornamento circa l'andamento dei ticker.

## 2. Alert Notifier System

- Tale servizio, al ricevimento di un messaggio dal topic “to-notifier” (di cui è consumer), invia un'email con i seguenti parametri:
  - **To:** l'indirizzo email specificato
  - **Subject:** il valore del ticker
  - **Body:** la condizione di superamento soglia, indicando se è stata oltrepassata la soglia superiore o quella inferiore.
- Anche qui il commit dei messaggi ricevuti viene effettuato manualmente, tenendo conto delle considerazioni fatte per il componente “Alert system”

## 3. Zookeeper

- Funge da coordinatore centrale per i broker Kafka. In particolare, si occupa di:
  - **Gestire i broker:** Tiene traccia dello stato di ogni broker Kafka nel cluster.
  - **Coordinare il cluster:** Sincronizza e coordina i broker per garantire la consistenza nella distribuzione dei dati e nel bilanciamento del carico.
  - **Memorizzare le configurazioni:** Conserva informazioni fondamentali come la configurazione dei topic, i dettagli delle partizioni e i leader per ciascuna partizione.
  - **Garantire robustezza:** Protegge il sistema da eventuali failure, utilizzando un meccanismo di elezione di un nuovo leader nel caso in cui si verifichi un fault

## 4. Broker Kafka

- Sono i componenti centrali del cluster Kafka che in questo caso è costituito da 3 broker. In questo modo riusciamo a garantire robustezza in caso di fault.
- Sono presenti due topic: “to-alert-system” e “to-alert-notifier”, ciascuno costituito da una singola partizione con tre repliche, che verranno assegnate ai tre broker.
- I broker comunicano con Zookeeper per ottenere informazioni sul cluster, come l'assegnazione delle partizioni e il coordinamento dei leader.